

Project Documentation: Parameterized Tri-State Data Driver

Digital Design Lab

Project Overview

This lab implements a parameterized **Tri-State Data Driver** in Verilog HDL [?]. A tri-state driver acts as a controlled buffer that routes input data to the output bus when enabled, and disconnects from the bus by driving high-impedance (Z) states when disabled [?].

This circuit component is fundamental in digital systems for bus sharing and bi-directional communications without causing contention.

Specifications & Functionality

- **Default Bus Width:** `WIDTH = 8` bits (Parameterized) [?].
- **Control Functionality** [?]:
 - When `data_en = 1'b1`: Output passes input data (`data_out = data_in`) [?].
 - When `data_en = 1'b0`: Output enters high-impedance state (`data_out = 'bz`) [?].

Port Summary

Port Name	Direction	Type	Width	Description
<code>data_in</code>	Input	wire	<code>WIDTH</code>	Input data vector [?, ?]
<code>data_en</code>	Input	wire	1	Tri-state driver enable signal [?, ?]
<code>data_out</code>	Output	reg / wire	<code>WIDTH</code>	Driver output bus [?, ?]

Table 1: Data Driver Interface Specifications

Critical Design Fixes & Analysis

Important Corrections Required

When analyzing the initial implementation of `driver.v`, two key discrepancies were identified between the source code, the specification, and the testbench:

1. **Port Bit-Width Mismatch (`data_en`):** The initial design declared `input [1:0] data_en` [?]. However, the testbench [?] and specifications [?] define `data_en` as a single-bit signal (`1'b0 / 1'b1`). Declaring it as 2 bits causes connection port mismatches during instantiation.
2. **Incorrect Disabled Output State:** The original code used `data_out <= 0` when disabled [?]. Driving 0 (logic LOW) actively pulls the bus down instead of disconnecting it. To model a true driver/buffer, it must assign high-impedance (`'bz`) [?].

Verilog Source Code & Line-by-Line Explanations

Corrected Design Module (driver.v)

Below is the corrected procedural block implementation for `driver.v`:

```
module driver #(
    parameter WIDTH = 8
)(
    input wire [WIDTH-1:0] data_in,
    input wire          data_en, // Corrected from 2-bit [1:0] to
    1-bit
    output reg [WIDTH-1:0] data_out
);

    always @(*) begin
        if (data_en) begin
            data_out = data_in;
        end else begin
            data_out = {WIDTH{1'bz}}; // Corrected to drive High-
            Impedance Z
        end
    end
endmodule
```

4.1.1 Design Code Breakdown

- `parameter WIDTH = 8`: Allows the module to be instantiated dynamically with any bus width, defaulting to 8 bits [?, ?].
- `input wire data_en`: Single-bit enable signal controlling driver operation [?, ?].
- `always @(*)`: Combinational process block sensitive to changes in any input signal (`data_in`, `data_en`).
- `{WIDTH{1'bz}}`: Replicated concatenation literal. Ensures that every single bit of the output bus is driven to high-impedance (Z) regardless of the configured `WIDTH` [?].

Alternative Continuous Assignment Approach

In Verilog, combinational tri-state logic can also be concisely modeled using a continuous assignment with a conditional ternary operator:

```
module driver #(parameter WIDTH = 8)(
    input wire [WIDTH-1:0] data_in,
    input wire          data_en,
    output wire [WIDTH-1:0] data_out
);
    assign data_out = (data_en) ? data_in : {WIDTH{1'bz}};
endmodule
```

Testbench Module (driver_test.v)

```

`timescale 1ns / 1ps

module driver_test;

    localparam WIDTH = 8;

    reg          data_en;
    reg [WIDTH-1:0] data_in;
    wire [WIDTH-1:0] data_out;

    // Instantiate Unit Under Test (UUT)
    driver #(
        .WIDTH (WIDTH)
    ) driver_inst (
        .data_en  (data_en),
        .data_in  (data_in),
        .data_out (data_out)
    );

    % Custom verification task
    task expect;
        input [WIDTH-1:0] exp_out;
        begin
            if (data_out != exp_out) begin
                $display("TEST FAILED");
                $display("At time %0d data_en=%b data_in=%b data_out=%b",
                    $time, data_en, data_in, data_out);
                $display("data_out should be %b", exp_out);
                $finish;
            end else begin
                $display("At time %0d data_en=%b data_in=%b data_out=%b",
                    $time, data_en, data_in, data_out);
            end
        end
    endtask

    // Stimulus Generation
    initial begin
        data_en = 1'b0; data_in = 8'hXX; #1 expect(8'hZZ);
        data_en = 1'b1; data_in = 8'h55; #1 expect(8'h55);
        data_en = 1'b1; data_in = 8'hAA; #1 expect(8'hAA);
        $display("TEST PASSED");
        $finish;
    end

endmodule

```

4.2.1 Testbench Breakdown & Key Verification Concepts

- **Case Inequality Operator (!=) [?]**: Standard inequality (!=) evaluates to unknown (X) if any bit contains Z or X. The case inequality operator != performs an exact structural comparison including Z and X states, which is mandatory when testing high-impedance logic [?].

- **Self-Checking Task (expect) [?]**: Encapsulates assertion logic, automatically terminating simulation via `$finish` upon encountering a mismatch [?].
- **Tri-state Stimulus Step [?]**:
 - At $t = 0$, `data_en = 0` and `data_in = 8'hXX` [?].
 - At $t = 1$, the `expect(8'hZZ)` task verifies that the output is high-impedance despite uninitialized/unknown input [?].

Simulation & Verification

Running Simulation with Synopsys VCS

To compile and execute the test suite in batch mode [?]:

```
vcs driver.v driver_test.v
./simv
```

Alternatively, to open GUI mode for waveform inspection [?]:

```
vcs driver.v driver_test.v -gui-access+rw
```

Expected Output Results

Upon applying the corrections, running the simulation produces the expected results [?]:

```
At time 1 data_en=0 data_in=xxxxxxx data_out=zzzzzzzz
At time 2 data_en=1 data_in=01010101 data_out=01010101
At time 3 data_en=1 data_in=10101010 data_out=10101010
TEST PASSED
```